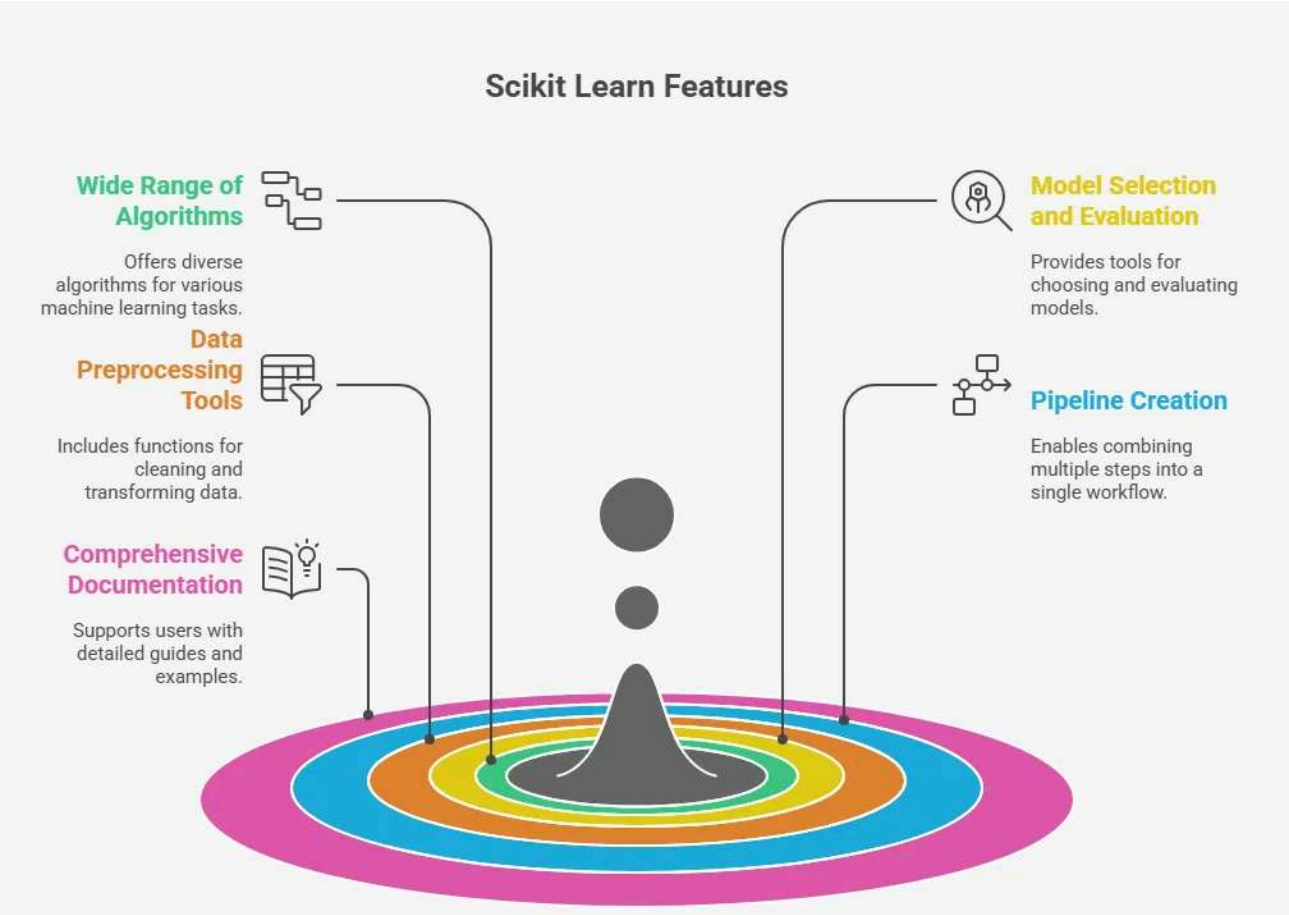
What is Scikit-learn?

Scikit-learn (sklearn) is a popular open-source Python library used for Machine Learning (ML). It provides simple and efficient tools for data preprocessing, classification, regression, clustering, model evaluation, and feature selection.

It is built on top of NumPy, SciPy, and Matplotlib Website: https://scikit-learn.org

Future of Scikit-learn

1. **Growing Demand in Machine Learning** – Scikit-learn will continue to be widely used as machine learning applications increase in different industries.
2. **Better Performance** – Future versions are expected to improve speed, efficiency, and memory usage.
3. **Integration with AI Tools** – Scikit-learn will continue to work well with AI and data science libraries such as NumPy, Pandas, Matplotlib, and TensorFlow.
4. **Support for New Algorithms** – More advanced machine learning algorithms and improved existing algorithms are expected to be added.
5. **Improved Data Processing** – Better tools for handling large datasets and preprocessing data are likely to be introduced.
6. **More Automation** – Future updates may include more automated machine learning (AutoML) features to simplify model building.
7. **Cloud and Big Data Support** – Scikit-learn is expected to improve integration with cloud platforms and big data tools.
8. **Growing Community** – A large developer community will continue to improve the library by fixing bugs and adding new features.
9. **Educational Importance** – Scikit-learn will remain one of the most popular libraries for learning and teaching machine learning.
10. **Wider Industry Use** – It will continue to be used in healthcare, finance, education, e-commerce, cybersecurity, and many other industries.



Installation

```
pip install scikit-learn
```

Import Scikit-learn

```
import sklearn

Check version

import sklearn
print(sklearn.__version__)
```

Main Modules of Scikit-learn

```
Main Modules of Scikit-learn
sklearn.datasets
sklearn.model_selection
sklearn.preprocessing
sklearn.linear_model
sklearn.tree
```

```
sklearn.ensemble
sklearn.neighbors
sklearn.cluster
sklearn.svm
sklearn.metrics
sklearn.naive_bayes
```

Loading Dataset

Example

```
from sklearn.datasets import load_iris iris = load_iris() print(iris.data)
```

Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42f
)
```

Purpose

Training Data → Used to train the model. Testing Data → Used to test the model.

Data Preprocessing

Standard Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

Min-Max Scaling

```
from sklearn.preprocessing import MinMaxScaler scaler = MinMaxScaler() X_scalec
```

Label Encoding

```
Label Encoding
```

Machine Learning Algorithms

1. Linear Regression

Used for predicting continuous values.

```
from sklearn.linear_model import LinearRegression model = LinearRegression() mc
```

Applications

House price prediction Salary prediction Sales prediction

3. Decision Tree

```
from sklearn.tree import DecisionTreeClassifier model = DecisionTreeClassifier(
```

Applications

Customer classification Medical diagnosis

Customer classification Medical diagnosis 4. Random Forest

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
```

Applications

Fraud detection Recommendation systems

5. K-Nearest Neighbors (KNN)

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier()
```

Double-click (or enter) to edit

Applications

Pattern recognition Image classification

Pattern recognition Image classification

6. Support Vector Machine (SVM)

```
from sklearn.svm import SVC
```

Applications

Face recognition Text classificatioApplications

7. Naive Bayes

```
from sklearn.naive_bayes import GaussianNB model = GaussianNB()
```

Applications

Spam filtering News classification

8. K-Means Clustering

```
from sklearn.cluster import KMeans
model = KMeans(n_clusters=3)
```

Applications

Customer segmentation Market analysis

Model Training

```
model.fit(X_train,y_train)
```

Prediction

```
prediction = model.predict(X_test)
```

Model Evaluation

Accuracy

```
from sklearn.metrics import accuracy_score accuracy_score(y_test,prediction)
```

Confusion Matrix

Mean Squared Error

```
from sklearn.metrics import mean_squared_error mean_squared_error(y_test,predic
```

R^2 Score

```
from sklearn.metrics import r2_score r2_score(y_test,prediction)
```

Hyperparameter Tuning

Grid Search

```
from sklearn.model_selection import GridSearchCV

Random Search

from sklearn.model_selection import RandomizedSearchCV
```

Pipeline

```
from sklearn.pipeline import Pipeline
```

Purpose

Combines preprocessing and model training into one workflow

Feature Selection

from sklearn.feature_selection import SelectKBest

Purpose

Selects the most important features.

Dimensionality Reduction

PCA

```
from sklearn.decomposition import PCA
```

Purpose

Reduces the number of features while preserving important information.

Saving Model import joblib

```
import joblib

joblib.dump(model,"model.pkl")

Load Model

model = joblib.load("model.pkl")
```

Common Scikit-learn Functions

Function Purpose train_test_split() Split data into training and testing sets fit() Train the model predict() Predict output score() Calculate model score accuracy_score() Measure classification accuracy confusion_matrix() Evaluate classification performance classification_report() Generate precision, recall, and F1-score StandardScaler() Standardize features MinMaxScaler() Normalize features LabelEncoder() Convert labels

into numbers GridSearchCV() Hyperparameter tuning cross_val_score() Cross-validation Pipeline() Create machine learning pipeline PCA() Dimensionality reduction

Applications of Scikit-learn

Machine Learning Artificial Intelligence Data Science Predictive Analytics Fraud Detection Medical Diagnosis Recommendation Systems Customer Segmentation Spam Detection Image Classification Text Classification Financial Forecasting

Scikit-learn vs TensorFlow

Scikit-learn TensorFlow Used for traditional machine learning Used for deep learning Easy to learn More complex Small and medium datasets Large datasets CPU-based CPU and GPU support Simple API Advanced API